



Detailed Methodology

This appendix outlines the detailed computational methodology employed to predict redox potentials of iron-sulfur proteins, adding information on script functions and input/output, naming conventions, and directory structures to allow better reproducibility and complement Chapter 3.

E.1 Data Collection

E.1.1 Data Collection Format

The data for this project was recorded in the following formats:

- An Excel sheet containing information on the proteins, their cofactors, and corresponding redox potentials. There is then one row per cofactor/redox potential.
- A folder containing the 3D protein structures from AlphaFold, as *.pdb* files..
- A folder containing the 3D protein structures from crystallography, if available, in the default *.ent* file format.

The Excel sheet is filled with the variables described in table E.1. Some extra notes to accompany this table:

- **Mutations** were all substitutions, written using single-letter amino acid codes, in the format {original amino acid}+{location in sequence}+{replacement amino acid}. If there are multiple mutations in the same protein, they are joined with an underscore.
 - For example, mutation entry D13G_V14G represents an Aspartic Acid in position 13 and a Valine in position 14, each being substituted by a Guanine.
- **Classification** examples include *electron transport* and *oxidoreductase*.
- **Cofactors** are written as: [Fe3+], [2Fe-2S], [3Fe-4S], [4Fe-4S].
- **Family** is based on the **cofactor**. Among others, we have:

- Rubredoxins, HiPIPs, Rieske proteins, Nitrogenases
- [3Fe-4S]Ferredoxins, [4Fe-4S]Ferredoxins
- [4Fe-4S]/[3Fe-4S]Ferredoxins, with one 4-Iron and one 3-Iron cofactor
- 2[4Fe-4S]Ferredoxins, with two 4-Iron cofactors
- 2[4Fe-4S]/[3Fe-4S]Ferredoxins, with two 4-Iron cofactors and one 3-Iron cofactor
- and so on...
- **Which cofactor** gives us a way to differentiate the cofactors in the case of 2[4Fe-4S]/[3Fe-4S]Ferredoxins and 2[4Fe-4S]Ferredoxins.
 - The **proximal** cofactor is more buried and has a lower (more negative) redox potential. It is closer to the electron donor in the electron transport chain.
 - The **distal** cofactor is less buried, has a higher redox potential, and is closer to the electron acceptor.
 - The **intermediate** cofactor is always the [3Fe-4S] cofactor in 2[4Fe-4S][3Fe-4S]Ferredoxins, and, as the name indicates, is usually located in between the two [4Fe-4S] cofactors, mediating the electron transport.

Table E.1: Data collection variables and descriptions

Variable	Description
UniProt ID	Protein's unique identifier from UniProt database
PDB	PDB ID for the protein crystal structure, if any
Name	Name of the protein as given in UniProt
Organism	Organism to which the protein belongs
Organism (alternative name)	Alternative/old name for the organism, if any
Strain	Organism strain, as mentioned in UniProt or on paper, if any
Mutation	'no' or the mutation
Classification	Main function of the protein
Family	Type of iron-sulfur protein
Cofactor	Type of iron-sulfur cluster
Which cofactor	If multiple cofactors in the same protein: 'proximal,' 'distal,' or 'intermediate'
pH	pH at which the redox potential was measured
Em	Redox potential
Reference – redox potential	DOI for redox potential paper
Reference – structure	DOI for crystal structure paper, if any
Resolution (Angstrom)	Crystal structure resolution, if any
Notes	Anything of relevance about the entry to note

E.1.2 Finding Redox Values and protein 3D Structures

UniProt database search I conducted the first phase of the search for iron-sulfur cluster proteins in UniProt, a freely accessible database of protein sequences and other functional information. Two filters were applied, joined by an “AND” operator:

1. *Function / Cofactor / ChEBI term*: “30408”
2. *Function / Biophysicochemical properties / Redox potential*: “mV”

The first filter identifies proteins with a specific chemical compound as their cofactor, given by the ChEBI term. ChEBI, short for Chemical Entities of Biological Interest, is a sort of dictionary of molecular entities, more specifically focused on “small” chemical compounds, a category which iron-sulfur clusters fall under. The term used in this search, **30408**, is for iron-sulfur clusters, described as “a unit comprising two or more iron atoms and bridging sulfur ligands,” [European Bioinformatics Institute, 2018]. This term does not include rubredoxins, because at this stage of my research, I did not yet know they existed; all my data for rubredoxins was subsequently extracted directly from scientific papers. Applying only this first filter yielded over 12,000 results, so to refine it more, I applied the second filter, which looks for the keyword “mV” in the redox potential property of a protein entry, which efficiently isolates the entries for which a redox potential is noted in UniProt. The final search query is then:

```
(cc_cofactor_chebi:“CHEBI:30408”) AND (cc_bpcp_redox_potential:mV)
```

Then, I moved my search to Scopus.

Scopus database search I applied the following two conditions, joined by an “AND” operator:

- Within the keywords, abstract, and title: “redox” AND “iron-sulfur” AND “mV”
- Within the keywords, abstract, and title: “ferredoxin*” OR “rubredoxin*” OR “rieske” OR “hipip*”

The asterisk represents any string of characters, to expand our search. The resulting search query is:

```
(TITLE-ABS-KEY( iron-sulfur ) AND TITLE-ABS-KEY( redox ) AND  
TITLE-ABS-KEY( mv ) AND (TITLE-ABS-KEY( ferredoxin* ) OR TITLE-ABS-KEY(  
rubredoxin* ) OR TITLE-ABS-KEY( rieske ) OR TITLE-ABS-KEY( hipip* )))
```

which yields 202 documents. Then, I checked the abstract to see if the “mV” values are indeed referring to redox potentials of the iron-sulfur proteins. If these conditions were fulfilled, I looked inside the paper and filled the Excel datasheet accordingly.

Downloading structures To find the proteins on UniProt, I searched for the organism name and protein name as given in the papers. It was usually pretty straightforward to find the right protein, and the 3D structure could subsequently be downloaded from the “Structure” section, and any other missing information on the Excel sheet could be recorded. To validate the structures, I:

- Looked to see if most of the AlphaFold structure was colored dark blue, indicating a high confidence prediction score.

- Compared the AlphaFold structure with the crystallography or NMR structure, if available, to see if they were similar.

Structure files were organized and named according to the following convention:

- **Wild types:** File name {UniProt ID}_WT_Structure.pdb, within a folder FeS_WildType_structures
- **Mutants:** File name {UniProt ID}_{Mutation}_Structure.pdb, within a folder FeS_mutant_structures_uniminimized

Saving references All the papers I read and extracted information from throughout this project were saved on Zotero, tagged with the UniProt ID(s) of the protein(s) they mention, and/or with other meaningful tags (like “crystal structure” or “ML”).

E.2 Mutant Structures

During the data collection step, some mutant entries were recorded containing 1 to 3 amino acid substitutions. Unfortunately, none of these entries have crystal structures available, and AlphaFold is not sensitive enough to these small sequence changes to provide accurate mutant predictions. Therefore, to obtain the structures of mutant entries, mutations were first induced in the wild-type AlphaFold PDB files, followed by energy minimization to ensure physically realistic conformations.

E.2.1 Tools and Packages for Energy Minimization

A combination of specialized software tools and Python packages is used to carry out energy minimization on the mutated protein structures. Each tool contributed to specific stages in the pipeline, from structural manipulation to physical optimization. Table E.2 summarizes the sequential steps and corresponding software tools used in this process.

To ensure reproducibility and avoid conflicts between software dependencies, separate virtual environments were created for different stages of the project. A virtual environment is an isolated workspace that contains its own installation of Python and libraries, enabling users to install and switch between different versions of software without conflicts. The environments were managed using **mamba**, a faster alternative to **Conda** when it comes to resolving dependencies. More information is provided in Table A.1, which summarizes the main virtual environments created for this project, their respective roles, and the key packages installed in each.

Detailed implementation of this pipeline, including code-level execution and file handling, is described in the following subsection.

E.2.2 Energy Minimization Detailed Pipeline

This subsection provides a technical description of the energy minimization pipeline used for the mutant protein structures. Each stage builds upon the tools introduced in Section E.2.1, implementing them via Python scripting or shell commands to automate a reproducible workflow.

Table E.2: Software tools and packages used for energy minimization

Step	Tool / Package	Role in Project
1. Mutation induction	PyMOL (API)	Used via its Python API to induce site-specific amino acid substitutions in AlphaFold-predicted protein structures. This allowed structural modifications directly in the PDB files.
2. Protonation assignment	PDB2PQR	Assigned protonation states based on pH-specific considerations. Generated PQR files needed for subsequent charge and electrostatics calculations.
3. Charge calculation	Base Python, pandas	Used to parse the PQR files and calculate the net charge of each protein, which determined the number of ions needed for system neutralization.
4. System preparation	AmberTools / tleap	Used to solvate the protein, add counterions, add missing Hydrogen atoms, and prepare the system using the AMBER force field. Output included topology files and coordinate files in AMBER format.
5. File format conversion	ParmEd	Converted AMBER-formatted topology and coordinate files into formats compatible with GROMACS (.gro and .top), ensuring interoperability between software tools.
6. Energy minimization	GROMACS	Performed the actual energy minimization using a steepest descent algorithm.
7. File format conversion	Base Python, subprocess	Used to convert minimized structures (.gro files) back to PDB format.
8. Structure extraction	Base Python	Used to parse the PDB files and remove the water atoms and ions around the minimized structure.
9. Reassign missing chain IDs	Base Python	Used to parse the PDB files and reassign amino acids in the minimized structure to chain 'A' for analysis.

Step 0: Mutation indices First, as discussed in section 2.3, the AlphaFold structure contains the entire, immature protein, while the protein sequence used by researchers typically has already undergone all post-translational modifications. This means that the mutation index extracted from the papers and indicated in the “Mutations” column of the Excel dataset is not correct in the AlphaFold PDB file. To address this issue, I created a new column on the Excel table, named “Mutations_AF,” which contains, like the original “Mutations” column, either ‘no’ or the mutation at the corrected index. To find this index in the full AlphaFold sequence, I looked at every mutant protein’s entry on UniProt, in the *PTM/Processing* section, and found the position of the “Chain” in the sequence. The index of the mutation in AlphaFold is then the index of the start of the chain plus the mutation index from the paper, minus 1 (to avoid counting the first residue of the chain

twice). An example is given in Figure E.1.

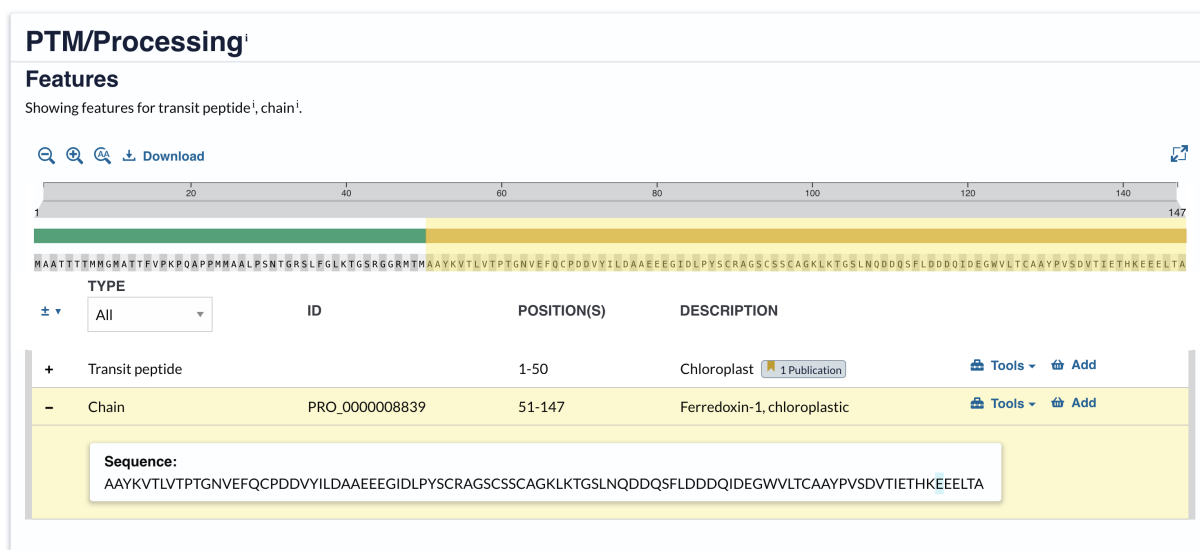


Figure E.1: Screenshot of entry P00221’s UniProt page in the *PTM/Processing* section. Aliverti et al. induced a mutation at position 92 in the mature protein, which is position 142 (51+92-1) (highlighted here in blue) in the AlphaFold sequence.

Step 1: Inducing mutations All mutations were induced using the script `mutate_proteins.py`, which takes as input:

- A folder containing the wild-type protein PDB files
- The Excel dataset, with “Mutations_AF” column

And gives as output:

- A folder containing the mutant PDB files, all named {UniProt ID}_{Mutation}.pdb
- A CSV file that logs whether the entry is skipped because it is a wild-type, or if a mutation is applied. For the latter, it records how many mutations, if they were successful, and logs the errors if not.

Step 2: Assigning protonation states Protonation states are assigned using `pdb2pqr`, installed in virtual environment `protonation-env`, with the script `assign_protonation.sh`, which is written in bash instead of Python for easier CSV parsing. It takes as input:

- The folder containing mutant PDB files, output of **Step 1**.
- The Excel dataset, with pH values

And gives as output:

- Protonated structure files, in `.pqr` format
- A CSV file that logs status information on the protonation assignment (success, or failure and error)

Step 3: Calculating protein charge The charge of the protonated structures is calculated in the script `generate_solvation_parameters.py`, which takes as input:

- The folder containing the protonated structure files, output of **Step 2**.

And gives as output:

- The solvation parameters: a CSV file with the charge of the protein, and the number of Na⁺ or Cl⁻ ions needed to neutralize the structure.

Step 4.a: tleap preparation The `tleap` program command files are generated for each mutant with the script `generate_tleap_inputs.py`, which takes as input:

- The folder containing the protonated structure files, output of **Step 2**.
- The solvation parameter CSV file, output of **Step 3**
- A template file for `tleap` commands, presented in Listings A.1

And gives as output, within a root folder `tleap_runs`, subfolders for each mutant, containing:

- The mutant structure, stripped of its hydrogen atoms and saved as a *.pdb* file.
- A customized `tleap` commands file, with a notable parameter of **10Å** of padding around the structure.

The hydrogens were removed from the structure because, when running the `tleap` commands, if the parametrization is not successful, it is often because of unrecognized hydrogen atoms. To avoid this issue altogether, they are removed before converting the file from PQR to PDB format. In **Step 4.b**, `tleap` will add them back according to its force field.

Step 4.b: System preparation The `tleap` commands are executed using AMBER, installed in the `ambertools_env` virtual environment, via the script `run_tleap_all.py`, which takes as input:

- The folder `tleap_runs`, output of **Step 4.a**.

And gives as output, in the corresponding structure subfolder within the `tleap_runs` folder:

- The solvated system files (protein structure with hydrogens, in a water box with ions)
- Log files capturing the `tleap` output

Step 5: AMBER to GROMACS file conversion The AMBER-format output files from **Step 4.b** are converted to formats supported by GROMACS using `ParmEd`, installed in the `parmed_env` virtual environment, via the script `convert_to_gromacs.py`, which takes as input:

- The AMBER topology (*.top*) and coordinate (*.crd*) files in the folder `tleap_runs`, output of **Step 4.b**.

And gives as output, still in the structure subfolders:

- GROMACS-compatible topology (*.top*) and structure (*.gro*) files, with suffix “_gro” to differentiate them from the AMBER topology files.

Step 6: Energy minimization Finally, the energy minimization is run using GROMACS, installed in the `gromacs_env` virtual environment, via the script `run_minimization.py`, which takes as input:

- The GROMACS topology (.top) and coordinate (.crd) files in the folder `t1eap_runs`, output of **Step 5**.
- A minimization parameters file (minim.mdp).

And gives as output, still in the structure subfolders:

- Energy minimized structures, with suffix “_minim”.
- Log files documenting the minimization process

Step 7: File conversion from GROMACS to PDB The energy minimization outputs a lot of files, but we are only interested in the file ending in “_minim.gro” which contains the final coordinates of our system after minimization. To use these for the rest of the report, we first need to convert them to PDB format, using the script `prepare_alphafill_inputs.py`, which takes as input:

- The directory containing the minimized structures.

And gives as output, in a new folder:

- PDB files of the minimized structures.

Step 8: Structure extraction The next step is to clean up the PDB file by removing all water atoms and ions added for minimization, leaving behind only the minimized structure. This is done with the `remove_water_box.py` script, which takes as input:

- The directory containing the minimized structure PDB files.

And gives as output, in a new folder:

- Cleaned up PDB files of the minimized structures.

Step 9: Reassigning chain IDs The energy minimization removed the chain IDs, which led to errors during the feature extraction step when the code would try to get the sequence of the protein. This was fixed by assigning all amino acids to chain ‘A,’ which is correct since all the structures involved are monomers. The script used is `add_chain_ids.py` script, which takes as input:

- The directory containing the minimized structure PDB files.

And gives as output, in the same folder:

- The PDB files with chain IDs.

E.3 Multimers

Some of the proteins in this dataset are only biologically relevant as multimers. Multimers are protein complexes that are composed of multiple protein subunits, monomers, which can be identical or different, and are non-covalently bound. In the data collected for this project, none of the papers mentioned whether the protein for which they measured the redox potential is in its monomeric or multimeric form. The large majority of the proteins in my dataset exist as monomers, but some of them, UniProt states, interact as homodimers, simply writing “Homodimer” in the *Interaction* section of the protein’s page. Cross-referencing this with PDB was also not helpful, as the “Biological assembly

1,” which should be the biologically relevant structure, would not be differentiated from the asymmetric unit. They would either both be homodimers, or both monomers.

To address this confusion, it was decided that:

- If a protein complex binds one cofactor per subunit, and that subunit is modeled correctly in AlphaFold, then only that subunit with its cofactor will be used in the dataset.
- If a protein complex binds one cofactor per subunit, and the subunit is not accurately modeled by AlphaFold, then the crystal structure of the complex will be used in the dataset instead, if available.
- If a protein complex binds one cofactor in the whole complex, then the AlphaFold structure(s) for the subunit(s) will be multimerized before implanting the cofactor and using it in the dataset.

Multimer assembly One entry in particular, UniProt ID P0DUV7, exists as a homotrimer. Its UniProt page’s *Interaction* section states that this protein “homotrimerizes to form a pseudohexamer with a large central pore, which is probably the binding site for the [4Fe-4S] center” [Pang et al., 2011]. To obtain this structure, I used ColabFold, a faster and more user-friendly platform for protein structure prediction using AlphaFold2 and AlphaFold-Multimer [Mirdita et al., 2022]. More specifically, I used the AlphaFold2_mmseqs2 Google Colab notebook available at ColabFold’s Github (<https://github.com/sokrypton/ColabFold>). In the query sequence, I pasted the complete protein sequence as found on UniProt, 3 times, separated by a colon each time. I named the job appropriately, and then clicked `Runtime / Run all`. No other settings were changed. The resulting homotrimer is shown in Figure E.2. We can see from the right structure that most of it is colored dark blue, indicating that AlphaFold is highly confident in its prediction, which validates our structure. From these images, we can clearly see the “large central pore,” which UniProt mentioned is likely the binding side of the cofactor.

Crystal structures Throughout the data collection, if a big discrepancy was found between the AlphaFold structure and the available crystal or NMR structure, the latter would be used instead. This happened in a few cases, where UniProt stated the protein interacts as a dimer, and the monomer predicted by AlphaFold would look nothing like the experimentally found homodimer structure. The following protein entries had their AlphaFold structure replaced by a crystal or NMR structure in the final dataset: P9WF43 (NMR), P00260 (crystal), Q9NZ45 (crystal), and Q8N5K1 (crystal).

E.4 Cofactor Implantation

Once all the protein structure files are downloaded, prepped, and converted to PDB format, they are ready to be implanted with their corresponding cofactor. For mutant structures, this step is done after the energy minimization rather than before due to the complexity and lack of defined force fields for Iron-Sulfur clusters in AMBER, which would make minimizing them extremely difficult.

In this section, a detailed implementation of the pipeline used to insert cofactors into the protein structures is described.

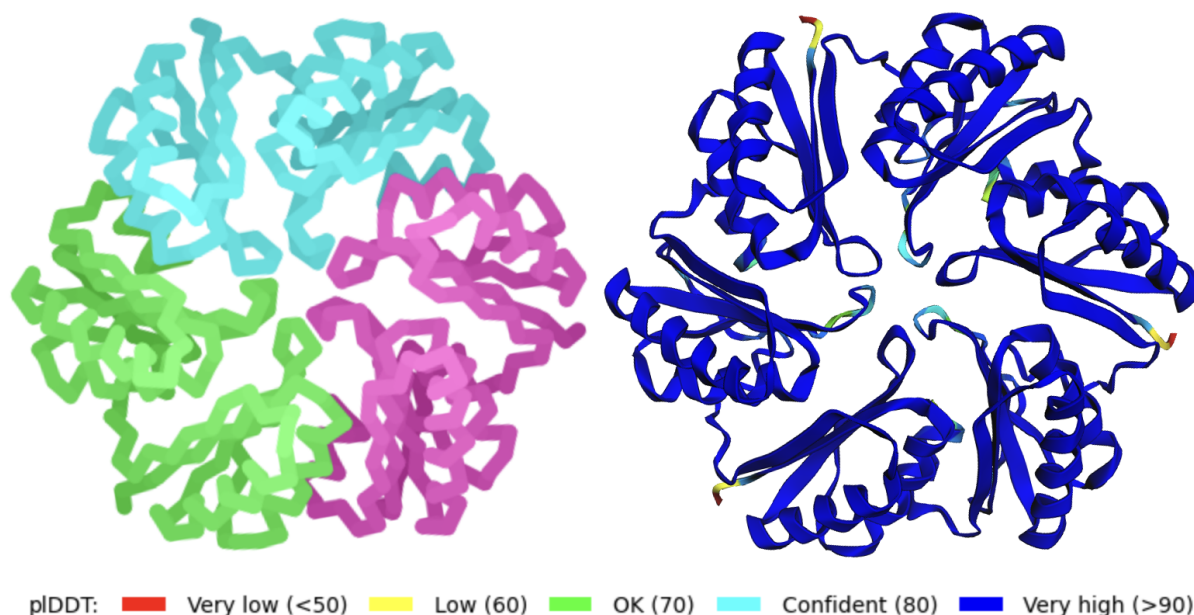


Figure E.2: Results of ColabFold’s prediction on homotrimerizing entry P0DUV7. The left image is the structure, colored by chain, showing clearly the trimerization. The right image is colored by AlphaFold’s prediction confidence level, according to the legend below the images.

E.4.1 AlphaFill transplants

All predicted proteins in the AlphaFold database lack coordinates for small molecules, ligands, and cofactors. The AlphaFill database is available at <https://alphafill.eu/>.

AlphaFill interface The AlphaFill database offers two options: “Show an entry based on Uniprot ID,” where a UniProt ID can be entered in a field, and “Create your own AlphaFill model,” which takes a structure PDB file as input. If structures with over 25% similarity in the sequence are found, AlphaFill returns the enriched structure. It is possible to look at results for 25%, 30%, 40%, 50%, 60%, and 70% identity. Of course, each time, only the structure at the highest identity possible (meaning, with results) is considered.

Downloading AlphaFilled structures As no API was found, I looked up each individual entry in my dataset manually. The entries that were not already present in the database were uploaded in the second field I mentioned. All output structures were downloaded into the same folder as *.cif* files, the default AlphaFill extension. However, not all of these structures were enriched with their corresponding ligands and cofactors: some were empty, while others had too many chemical compounds added.

(Overly) Enriched structures In the latter case, AlphaFill gives the option of selecting which compounds to keep in the final structure, so I would select just the compounds I knew are present in the protein, as stated in UniProt (usually just the Fe-S cofactor, but sometimes, also a Zinc ion or other small molecule), before downloading the structure. In the cases where only one compound is left on the structure, AlphaFill gives the option to “optimize” it using an energy minimization technique called YASARA [Hekkelman et al.,

2023]. If this optimization was available, and if it did not run into an error, I would download the optimized structure.

Empty structures Structures with no added compounds were downloaded with a “_empty” suffix in the file name. I downloaded all the structures, regardless of whether they were enriched by AlphaFill or not, to ensure consistency and completeness in my dataset.

To prepare these structures for the rest of the project, they needed to be converted from *.cif* to *.pdb* format. This is done using *gemmi*, a package which I downloaded into my *alphafill-env* virtual environment, and done via the Python script *convert_cif_to_pdb.py*. All structures were initially saved in a folder named *alphafill_cif*, and the converted files were saved into a new folder *alphafill_pdb*.

E.4.2 “Manual” transplants

After the initial search on AlphaFill, more than half the structures in my dataset were still empty. This meant another method of cofactor implantation was necessary. I called this part of my project the “manual” transplants, though the process is sometimes automated to transplant to multiple proteins at once. All transplants were done through Python scripts, using the PyMOL API. There are four phases to this part of the project:

1. Cofactor transplants into wild-type protein structures from their crystal or NMR structures (backbone alignment).
2. Cofactor transplants into mutant protein structures from their wild-type counterparts (backbone alignment).
3. Cofactor transplants into wild-type proteins from structurally similar protein donors (backbone and local alignment).
4. Cofactor transplant exceptions.

Transplants from crystal or NMR structures

For this phase of the cofactor transplants, I moved all the downloaded crystal or NMR structures to a folder called *crystal_donors*. I then converted them to PDB format from their native *.ent* format using the Python script *convert_ent_to_pdb.py*, and saved the converted files to a folder *crystal_donors_pdb*. The transplant itself is executed using the Python script *transplant_cofactors_from_crystal.py*. The donor file is *crystal_donors_pdb* and the target file is *alphafill_pdb*. This script parses the file names in each folder and tries to find a match for the target file name in the donor folder, if the target file has a “_empty” suffix. If a donor is found, it uses backbone alignment to line up both structures, then extracts the cofactor from the donor and implants it into the target, and saves the resulting structure. At every step, error handling logs if an element is not found by the code, and moves on to the next file in the folder. Since the donors concerned only wild-type proteins, the outputs were saved with a “_transplanted” suffix into a folder named *alphafilled_wild_types*. For completeness and for use in the next phase of this transplant step, the wild-type structures that had been correctly completed by AlphaFill were also manually copied into this folder.

Transplants from wild-type counterparts

Transplants into minimized mutant structures from their wild-type counterparts are done with the script `transplant_cofactors_mutants.py`, which uses similar logic as the `transplant_cofactors_from_crystal.py` script. For the donors, it takes the `alphafilled_wild_type` folder, and the target folder is the `input_structures` folder initially created for AlphaFill, containing the wild-type protein structures and the minimized mutant structures. The script parses the file names and looks specifically for those with “minimized” in the name. For each file found that fulfills this condition, it finds the corresponding wild-type donor in the donor folder and aligns the two structures via backbone alignment, then transplants the cofactor. The resulting structures are saved in a folder `alphafilled_mutants_manual` with naming convention `{UniProt ID}_{mutation}_transplanted.pdb`.

Transplants from structurally similar donors

After both previous steps were done, many protein structures still remained without a cofactor. For these, structurally similar protein donors had to be visually inferred to be their donors. This was done in PyMOL, where I would load the target protein, and then load a few proteins to try as donors. I would start with proteins from the same family, as those usually have similar structures. As such, all rubredoxins were filled by another rubredoxin (the first of which was filled by its crystal donor), all HiPIPs were filled by another HiPIP, and so on. The script used to execute the transplant itself is `manual_cofactor_transplant.py`, which takes a target and donor file as input, and has a setting `USE_GLOBAL_ALIGNMENT`. If set to `True`, the target and donor are aligned by their backbones. If set to `False`, it takes specific residues to align in both structures, for a local alignment. In a protein’s UniProt page, the “Function” section has a “Features” subsection which lists the position (residue numbers) of the binding sites for the ligands and cofactors of the protein. When these were given, which was almost always the case, local alignment was always used. If they were not, then global alignment was used. The cofactors to extract are also passed to `COFACTOR_RESN` as the 3-letter identifiers used by the PDB. The setup for this script is shown in Listing E.1. It is important to note that when the donor structure has multiple of the same cofactor, if selected, they will all be copied into the target structure. So, for example, if a target just needs one SF4 cofactor, the donor also ideally only has one SF4 cofactor.

Listing E.1: Configuration and first lines of the `manual_cofactor_transplant.py` script

```

1  # === User Config ===
2  # Input files
3  DONOR_FILE = "donor_file_path"
4  TARGET_FILE = "target_file_path"
5  OUTPUT_FILE = "output_file_path"
6
7  # Toggle for alignment strategy
8  USE_LOCAL_ALIGNMENT = True # Set to False to use backbone alignment
9
10 # Only used if USE_LOCAL_ALIGNMENT is True
11 DONOR_SELECTION = "resi 9+12+15+19"
12 TARGET_SELECTION = "resi 38+41+50+54"
13
14 # Cofactor residue names to extract
15 COFACTOR_RESN = ["SF4"] # ["FES", "SF4", "FE", "FE2", "F3S"]
16
```

```

17 # Log file path
18 LOG_FILE = "CSV_log_file_path"
19
20 # === Start PyMOL
21 cmd.reinitialize()
22 cmd.load(DONOR_FILE, "donor")
23 cmd.load(TARGET_FILE, "target")
24
25 # For the CSV log file
26 alignment_type = "Local" if USE_LOCAL_ALIGNMENT else "Global"

```

Transplant exceptions

For some structures, it was not as straightforward to implant a cofactor. Namely, the homotrimerized entry P0DUV7 had no clear donor and required modeling and coloring of the coordinating residues in PyMOL (using the commands `show sticks` and `color`) to manually implant the cofactor directly in the PyMOL application and visually assert its position.

Another entry, P00209, was initially filled by its crystal structure with a [3Fe-4S] cofactor, but one paper had inserted an extra Iron atom into the cofactor to transform it into a [4Fe-4S] cofactor for which they calculated the redox potential [Moreno et al., 1994]. For this entry, the bare structure file had to be duplicated to then manually implant a [4Fe-4S] cofactor. The two files are named P00209_F3S.pdb and P00209_SF4.pdb.

Lastly, entry P18187_P238C's mutation caused the intermediate [3Fe-4S] cofactor to turn into a [4Fe-4S] cofactor [Rousset et al., 1998]. So, while the initial minimized structure had two [4Fe-4S] cofactors and one [3Fe-4S] cofactor implanted into it, like with the other mutated P18187 entries, this one had to be changed manually by deleting the [3Fe-4S] cofactor directly in the PyMOL application, and choosing a new donor for the [4Fe-4S] cofactor, implanting it via `manual_cofactor_transplant.py`.

E.4.3 Validation and cleanup

For every cofactor transplant, the PyMOL-calculated root mean square deviation (RMSD) was recorded in a separate log file corresponding to each transplant script. Every structure was verified visually in PyMOL to see if the correct and right number of cofactors were implanted in each structure. For structures where the transplant RMSD was larger than 2Å, if possible, I would show the coordinating residues on the structure as sticks and try to manually rearrange the cofactor's position to fit better.

Once all the protein structures are complete and validated, they are ready for the next step of the project, the feature extraction.

E.5 Feature Extraction

My feature extraction code was adapted from the methodology of Galuzzi et al (2022). My approach extends their framework to iron-sulfur proteins by incorporating iron-sulfur cluster-specific structural features while maintaining their core concept of radius-based local feature extraction on top of protein-wide feature extraction. For the local feature extraction, while Galuzzi et al.'s code calculates features both within a certain radius of the cofactor's barycenter and within a certain radius of each atom in the cofactor, my

code only looks at the former, since iron-sulfur clusters are a lot more densely packed and small than the FAD or FMN cofactors found in flavoproteins.

The features in this project were extracted for distances ranging from 1Å to 16Å from the cofactor barycenter, in steps of 1, which then yields 16 datasets per run of the feature extraction script. These distances were chosen for thoroughness—the Fe-S cofactors having a diameter of around 4Å, and residues beyond 12Å being unlikely to affect the cluster’s redox potential [Terranova, 2024], this range of radii would encompass, with a safe margin, all possible relevant interactions.

The feature extraction is done via the script `extract_features.py`, which imports functions from the supporting scripts `utils.py` and `utils_structure.py`. All scripts are run within the `redox-env` virtual environment (see Table A.1).

The following subsections describe the feature extraction process in detail.

E.5.1 Feature Categories

Molecular descriptors considering only the region around the cofactor are subsequently referred to as “Bar region features” or simply “Bar features” and are prefixed in the extracted dataset with “`Bar.`”, while those considering the whole protein are prefixed with “`Protein.`”. These prefixes only apply to the first two of the four feature categories generated by the feature extraction pipeline, as described below:

Amino acid counts First, the amino acids within each region (Bar region and whole protein) are counted. These features are prefixed with “`.count`”, and since there exist 20 amino acids, this step of the feature extraction yields **40 features** (20 counts for each region). The amino acids are written with their 3-letter codes, yielding feature names such as: `Bar.count.ALA` (number of alanines within the specified radius around the cofactor barycenter) and `Protein.count.VAL` (number of valines in the entire protein).

Properties and means Values of 28 different physicochemical properties for all amino acids (described in Table A.2) are then calculated for each region. This table, `tableAmm.txt`, is also taken from Galuzzi et al. The final dimensions of the table are 28×20 , or rather 30×20 including the amino acid single-letter and 3-letter codes. The features prefixed by “`.prop`” are the sum of the property values for all the amino acids in the region considered, and the features prefixed by “`.mean`” are the property value divided by the total number of amino acids in that region. This extraction step yields 28×4 features, so **112 features**.

The final feature names are, for example: `Bar.mean.Hydrophobicity` (average hydrophobicity value in the Bar region considered) and `Protein.prop.P(helix)` (tendency of the amino acids in the entire protein to adopt a helix secondary structure).

Structure features The features referred to as “Structure features” include **4 features** whose names are prefixed by “`struct.`”:

1. `burial_depth`: calculated as the minimum distance from the cluster barycenter to the protein surface, modeled as a convex hull using SciPy’s `ConvexHull` function, in Angstrom.
2. `iron_count`: the number of iron atoms in the cofactor.

3. `is_hipip`: a binary indicator of whether the protein is a HiPIP (1 if yes, 0 otherwise).
4. `is_rieske`: a binary indicator of whether the protein is a Rieske protein (1 if yes, 0 otherwise).

For the binary indicators to be assigned, two lists of Rieske and HiPIP UniProt IDs are passed to the feature extraction script.

Sequence features These features are prefixed by “`seq.`” and are generated using PyBioMed’s Composition-Transition-Distribution (CTD) module. This module calculates descriptors for seven physicochemical properties of amino acids, classifying each amino acid in one of 3 predefined groups, based on literature-derived scales. These properties, groups, and their descriptions can be found in Table A.3. Unlike Galuzzi et al., who only use the `CalculateC` (calculate composition) function that returns the fraction of amino acids in each group, I also employed the `CalculateT` (calculate transition) function. This function returns the normalized counts, or frequency, of the transition between amino acids of different groups along the sequence, for each property. Respectively, both these functions return $7 \text{ properties} \times 3 \text{ groups} = 21 \text{ features}$, and $7 \text{ properties} \times 3 \text{ transitions} = 21 \text{ features}$, so $2 \times 21 = \mathbf{42 \text{ features}}$ in total. Examples of feature names in this category include: `seq.CTDT__HydrophobicityT12` (frequency of transitions between groups 1 and 2 qua hydrophobicity) and `seq.CTDC__SolventAccessibilityC2` (fraction of amino acids in group 2 qua solvent accessibility).

All of these molecular descriptors together, plus the pH value at which the redox potential was measured, give **199 features** for each cofactor.

E.5.2 Feature Extraction Pipeline

The following paragraphs will detail the pipeline of the feature extraction script, `extract_features.py`.

All validated protein structures with their implanted cofactors were copied into a folder `final_structure_dataset`. I cleaned the file names using Python to remove all the suffixes, leaving the file names as `{UniProt ID}_{mutation, if any}.pdb`. The script used is `clean_structure_file_names.py`, which has two implementations: one where files with the same name are overwritten, and one without overwriting. This folder serves as the input for the feature extraction script.

The script then, for each radius listed:

1. Iterates over all the structures in the folder; for each structure, it calls the `get_all_FeS_clusters` function from `utils_structure.py` to identify all the Fe-S clusters in the protein.
2. Calculates the burial depth of the cofactors found.
 - In the case that there are two [4Fe-4S] cofactors, it labels them either “proximal” or “distal” according to burial depth—the proximal cluster being more buried than the distal one.
 - If there are two [4Fe-4S] cofactors and one [3Fe-4S] cofactor, the same convention is followed, only the [3Fe-4S] cofactor is then labeled “intermediate.”

- It handles the special case of entry P18187_P238C, where the [3Fe-4S] cofactor was turned into a [4Fe-4S] cofactor by the mutation, by measuring the pair-wise distance between the 3 cofactors, and labeling the middle one “intermediate.”
3. Labels all the cofactors found as {UniProt ID}_{mutation, if any}_{Cofactor PDB code}_{label (proximal, distal, intermediate), if any}. This is the cofactor ID.
 4. For each cofactor ID, runs the `extract_features` function, which:
 - (a) Parses the structure PDB file to extract the sequence.
 - (b) Extracts CTD features.
 - (c) Counts amino acids.
 - (d) Computes properties and their means according to `TableAmm.txt`.
 - (e) Adds the structural features.
 5. Returns the feature CSV files, named `features_r{radius}_final_{datetime stamp}.csv`, as well as log files.
 6. Returns metadata files for each radius with columns `structure_id`, `cofactor_id`, `cofactor_type`, `burial_depth`, and `radius`.

Before the pH and redox potential can be added to the features, one of the metadata files is used to add a `cofactor_id` column to the Excel data sheet, by joining the “UniProt” and “AF_mutation” columns in the latter with the `structure_id` column in the former. The folder containing the feature files and the updated Excel sheet are then passed to the script `merge_with_ph_em.py`, which matches the `cofactor_id` column in the feature files to the one in the Excel sheet to enrich the feature files with the redox potential and pH values. These files are returned in a separate folder, with a `_with_ph_em` suffix.

E.5.3 Dataset Division

To test the impact of different combinations of cofactors and features in the dataset, it was decided to divide the feature datasets into subsets, using the `create_feature_subsets.py` script. Separate datasets were then created for the 3 cofactor groups: all cofactors, only [4Fe-4S], only [2Fe-2S]. These groups were chosen since the [4Fe-4S] and [2Fe-2S] cofactors represent the largest subsets of the collected data. Then, for each cofactor group, 3 feature sets were created, containing: all features, only bar features, and only protein features. The structure features (with “`struct.`” prefix) and the pH and Em columns were kept in all the data subsets. The subset CSV files are named: `features_{radius, if relevant}_{cofactor (sub)set}_{feature (sub)set}.csv`. The resulting directory structure is shown in Section A.4.

Dividing the feature files into different subsets marks the last stage of the feature extraction phase, and the data is now ready to be run through the Machine Learning training pipeline, the implementation of which will be described in detail in the following section.

E.6 Model Training Pipeline

This section describes the methodology employed for training machine learning models to predict the redox potential of iron-sulfur proteins, based on the computational framework

adapted from Galuzzi et al. for flavoproteins [Galuzzi et al., 2022]. The training pipeline, implemented in Python, integrates data preprocessing, nested cross-validation, model training, performance evaluation, and feature importance analysis. The pipeline was designed to systematically evaluate multiple machine learning models across a range of spatial radii (1 Å to 16 Å) around iron-sulfur cofactors, capturing microenvironmental features influencing redox potential. The pipeline was implemented using the Scikit-learn library [Pedregosa et al., 2011], with additional feature importance analysis conducted using SHAP (SHapley Additive exPlanations) [Lundberg and Lee, 2017] when available. The following subsections detail each component of the pipeline.

E.6.1 Data Preprocessing

The input features are loaded from CSV files containing the extracted molecular descriptors and the target variable (redox potential, E_m , in millivolts). Preprocessing steps included:

- **Feature Selection:** Non-numeric columns are excluded, and features with zero variance across samples are removed using `VarianceThreshold`. Highly correlated features (Pearson correlation > 0.95) are also eliminated to reduce redundancy.
- **Missing Value Imputation:** Missing values are filled with the median of the respective feature.
- **Scaling:** For models requiring normalized inputs, features are standardized using `StandardScaler` to ensure zero mean and unit variance.

A crucial detail is that the preprocessing is performed within each cross-validation fold to prevent data leakage, ensuring parameters are derived solely from training data.

E.6.2 Nested Cross-Validation (CV)

A nested cross-validation strategy is used for hyperparameter optimization and performance evaluation. It is repeated 10 times with varying random seeds to assess generalization performance. The outer loop employs 5-fold KFold cross-validation, while the inner loop uses 3-fold KFold cross-validation and `GridSearchCV` to try all hyperparameter combinations and optimize them based on *negative* mean absolute error (MAE), since lower MAE is better. Since there is one new model instantiated and trained every outer CV fold, and the entire nested CV is repeated 10 times, the final model performance metrics are the average over $5 \times 10 = 50$ models. This structure ensures unbiased performance estimates by isolating hyperparameter tuning from final evaluation.

For feature data containing radius-specific features, this nested CV is performed for each combination of radius and model, using the ML training script `ml_training.py`. For the 3 feature data CSV files that are radius-independent (all cofactors, only SF4, and only FES “protein features only” data), this nested CV is performed for each model-dataset combination, in the `ml_training_protein.py` script.

E.6.3 Model Selection and Training

Seven machine learning models are trained, as mentioned in section 2.5: Linear Regression, Elastic Net, Support Vector Regression (SVR), Gaussian Process Regression (GPR), K-Nearest Neighbors (KNN), Random Forest, and Extreme Gradient Boosting(XGB).

Each model is integrated into a **Scikit-learn** Pipeline, incorporating preprocessing steps (scaling and feature selection) as needed. Table E.3 summarizes the models, their hyperparameter grids, and preprocessing requirements.

Feature selection is implemented within each outer CV fold to avoid overfitting by passing too many features to the models. For models needing feature selection (Linear Regression, Elastic Net, SVR, GPR, KNN), f -regression scores are used to measure the linear dependency between each feature and the target variable, in the end selecting only the top $\min(50, n/2)$ features, where n is the number of features in the training set. This means that for all training sets for these models needing feature selection, no more than 50 features are kept. The other two models, Random Forest and XGB, do not need feature selection as they already have built-in feature selection mechanisms through their training process, and are generally more robust to irrelevant features.

The best model for each radius/dataset is saved as a pickle file for further analysis after all iterations of the nested CV. Then, SHAP values are computed and graphs are generated for the top 3 models (for radius-independent datasets) or the top 3 model-radius combination (for the radius-dependent datasets). These values are subsequently analyzed using the script `find_recurrent_shap_features.py` as described in Subsection E.8.6.

E.7 Performance Evaluation and Interpretation

Model performance is assessed using multiple metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 score, Spearman and Pearson correlations, and Explained Variance. These metrics are computed for each fold and repeat, with means and standard deviations reported to quantify performance stability. Learning curves and prediction analysis plots (scatter, error distribution, and residual plots) are generated for each model and radius to visualize training dynamics and prediction quality.

For feature importance, SHAP analysis is performed on the top 3 performing models (based on MAE) across all radii. SHAP values are calculated using **TreeExplainer** for tree-based models (Random Forest, Gradient Boosting) and **KernelExplainer** for the others, analyzing feature contributions separately for protein-wide, local (Bar region), and all features. Summary plots and feature importance rankings are saved for the top 20 features per group.

E.8 Results Analysis and Visualization

The analysis and visualization of the iron-sulfur protein dataset and its machine learning (ML) predictions are conducted using a suite of custom Python scripts, tailored to handle both radius-dependent and radius-independent data. This section outlines the methodologies employed to process, analyze, and visualize the results, ensuring a comprehensive evaluation of dataset characteristics and model performance.

E.8.1 Dataset Statistical Analysis

Dataset statistics are computed using the `dataset_statistics.py` script, which analyzes structural and experimental data from PDB files, feature CSV files, and metadata Excel files. The script calculates protein size metrics (residue counts, 3D dimensions, and convex hull volumes) using the BioPython PDB parser. Cofactor type distributions, protein

family proportions, burial depths, redox potential statistics, pH values, mutation frequencies, and protein coverage are also plotted in various forms. Statistical measures (mean, median, standard deviation) are calculated, and visualizations (histograms, boxplots, and pie charts) are generated using Matplotlib and Seaborn, saved as high-resolution PNG files in a designated output directory. Detailed statistics are exported as CSV and JSON files for further analysis.

E.8.2 ML Training Result Interpretation

Dedicated scripts, `interpretation_protein.py` and `interpretation.py`, were developed to interpret and visualize ML training results from both radius-independent (processed with `ml_training_protein.py`) and radius-dependent (processed with `ml_training.py`) datasets, respectively. The script takes as input the complete output folder from the ML training script, loads the results from `ml_results.json`, and performs the following analyses:

- **Top Model Identification:** The top five models are ranked by mean absolute error (MAE), with performance metrics (MAE, R^2 , and standard deviations) reported for each radius or overall, depending on the dataset type.
- **Model Stability Analysis:** Stability is evaluated for radius-dependent data by computing cross-validation standard deviation and approximate prediction ranges across radii, visualized in dual-panel plots using Matplotlib.
- **Comprehensive Visualization:** A four-panel plot is created for radius-dependent data, displaying MAE and R^2 versus radius, optimal radius per model, and MAE distribution across all radii, excluding LinearRegression to avoid skewing color scales.
- **Performance Comparison:** Model performance on the radius-independent data is visualized for MAE and R^2 with side-by-side bar charts, with and without Linear Regression to avoid skewing scales.
- **Literature Comparison:** Model performance is benchmarked against a literature MAE of 36 mV for flavoproteins [Galuzzi et al., 2022], calculating percentage improvement.

A text-based report (`analysis_report.txt`) is generated, summarizing top models and performance per radius. Results are stored in an output directory named after the input directory and “_interpretation” suffix, as PNG visualizations.

E.8.3 Mann-Whitney Tests

Mann-Whitney U rank tests were performed using the `mann_whitney_tests.py` script, after copying all ML result CSV files, `ml_output_summary.csv`, from each dataset into an input directory, with naming convention `{dataset}_ml_output_summary.csv` (e.g., `complete_dataset_ml_output_summary.csv` and `FES_bar_ml_output_summary.csv`). Results are saved in a .txt file, `mann_whitney_results.txt` in the input folder.

E.8.4 Comparative Heatmap Generation

Comparative heatmaps are generated using `compare_datasets_heatmaps.py` for radius-independent data and `generate_heatmaps_complete.py` for radius-dependent data, addressing limitations in the ML training scripts’ visualizations (e.g., LinearRegression skew-

ing color scales and lack of model-specific heatmaps). For radius-independent data, `compare_datasets_heatmaps.py` creates heatmaps with dataset types (All cofactors, FES, SF4) on the x-axis and models on the y-axis, using MAE, RMSE, and R^2 metrics. For radius-dependent data, `generate_heatmaps_complete.py` produces dataset-specific heatmaps (radii on x-axis, models on y-axis) and model-specific heatmaps (datasets on y-axis, radii on x-axis), excluding LinearRegression from color range calculations to ensure consistency across all radii. Visualizations are saved as PNG files in designated output directories, leveraging Seaborn and Matplotlib for heatmap rendering.

E.8.5 Aggregated Results Analysis

To consolidate and compare ML results across multiple datasets, two dedicated scripts were developed: `aggregate_radius_dependent.py` for radius-dependent data and `aggregate_radius_independent.py` for radius-independent data. These scripts aggregate results from multiple output folders, each containing `ml_results.json` files, and generate comprehensive visualizations and reports.

For radius-dependent data, `aggregate_radius_dependent.py` processes results from six datasets: `complete_dataset` (all cofactors with both radius-dependent and radius-independent features), `all_cofactors_bar` (all cofactors, radius-dependent only), `FES_all`, `SF4_all`, `FES_bar`, and `SF4_bar`. The script performs the following:

- **Data Aggregation:** Collects performance metrics (MAE, R^2 , and standard deviations) across all datasets and radii, excluding LinearRegression to avoid skewing analyses.
- **Stability and Trend Analysis:** Computes mean MAE standard deviations per model and radius for stability, and performs linear regression on MAE versus radius to assess performance trends, reporting average slopes and p-values.
- **Visualizations:** Generates stability heatmaps (with and without LinearRegression), MAE distribution collages in 2x3 (with `complete_dataset`, `FES_all`, `SF4_all` in the first row; `all_cofactors_bar`, `FES_bar`, `SF4_bar` in the second row) and 3x2 layouts, and performance trend collages (MAE and R^2 versus radius) in 2x3 and 3x2 layouts for report and appendix datasets. A separate legend image is saved for model color mapping.
- **Reporting:** Outputs a text report (`aggregated_report.txt`) summarizing optimal model performance per dataset and radius, stability metrics, and trend analyses.

Visualizations are saved as high-resolution PNG files in an `aggregated_interpretation` directory, with a log file (`aggregation_log.txt`) capturing processing details.

For radius-independent data, `aggregate_radius_independent.py` aggregates results from datasets such as `all_cofactors_protein`, `FES_protein`, and `SF4_protein`, which use only radius-independent features. The script performs the following:

- **Data Aggregation:** Collects performance metrics (MAE, R^2 , and standard deviations) from `ml_results.json` files across all relevant datasets.
- **Top Model Selection:** Identifies the top five models by MAE, reporting performance metrics for each dataset.
- **Reporting:** Generates a text report (`aggregated_report.txt`) summarizing the top models and their performance, saved in an `aggregated_interpretation` directory, with a log file (`aggregation_log.txt`) for processing details.

Both scripts utilize Matplotlib and Seaborn for visualization and Pandas for data manipulation, ensuring consistent output formats and compatibility with the ML training pipeline.

E.8.6 SHAP Graph Analyses

To elucidate the key molecular and structural features influencing the performance of machine learning models for different datasets, SHAP (SHapley Additive exPlanations) values were computed to quantify feature importance for the top 3 models or model-radius combinations, selected based on their predictive performance measured by MAE, for each dataset subsets. The ML training scripts output these SHAP values as CSV files with two columns (features, SHAP importance score) and as graphs. Since there are 9 dataset subsets and 3 graphs for each, 27 graphs are generated for the entire dataset.

A Python script, `generate_shap_table.py`, was developed to automate the identification of recurrent features and generate a structured output. The script, implemented using the `pandas` library, processes the ML output SHAP CSV files for one specific dataset subset, stored in a directory. The script extracts the top N features (default $N = 5$) based on SHAP importance values from each of the CSV files. Recurrent features are defined as those appearing in at least M combinations (default $M = 3$), with both N and M configurable to allow flexibility in analysis (e.g., top 10 features or recurrence in at least 2 combinations). The script outputs a CSV file in the `output/` directory, with columns for Feature, Description, and binary indicators (`or`) for recurrence in all the CSV files. Feature descriptions are maintained in a dictionary within the script, mapping each feature to its physicochemical or structural significance (e.g., `seq.CTDC__HydrophobicityC3` as "Fraction of amino acids in hydrophobicity group 3 (hydrophobic)").

I ran this script 9 times to obtain all my data for the tables in Section 4.6. I focused on the top 5 features in all the SHAP CSV files, and filtered out those appearing in at least 2 of the 3 files. I do this for the subsets of all features, only bar features, and only protein features, for each of the cofactor subsets. So, the features that are filtered out by the script are added to a table (one table per cofactor subset), and a tick in the feature subset column marks whether that feature was recurrent in the feature subset dataset.

E.8.7 Results Availability

Results are systematically stored in structured output directories. Training outputs include JSON files (`ml_results.json`), CSV summaries (`ml_results_summary.csv`), and serialized best models as pickle files. Grid search parameters are saved as JSON (`grid_search_parameters.json`, `best_parameters_summary.json`). Visualizations encompass performance plots, SHAP summaries, feature importance evolution, model stability analyses, comprehensive comparisons, heatmaps, and aggregated collages, all saved as high-resolution PNG files. Analysis and aggregation reports are saved as `analysis_report.txt` and `aggregated_report.txt`, ensuring accessibility for further review.

E.9 Supercomputer Usage

All the ML training scripts were run using DelftBlue, the TU Delft’s supercomputer. I first recreated my `redox-env` virtual environment on DelftBlue, according to DelftBlue’s

documentation (<https://doc.dhpc.tudelft.nl/delftblue/howtos/conda/>), and using the Windows/Linux-adapted `portable_environment.yml` file I generated from my `redox-env`. The resources I asked of DelftBlue are shown in Listing E.2. Running the ML training loop on all scripts took about 23 hours.

Listing E.2: Computational Resources Requested From DelftBlue

```
1 --partition=compute
2 --ntasks-per-node=1
3 --cpus-per-task=16
4 --mem-per-cpu=3G
5 --time=24:00:00
```

E.10 Code Availability

The custom Python scripts developed for this study are available to support reproducibility and transparency. These scripts implement the statistical analysis, ML training result interpretation, and heatmap generation described in Sections E.8. The code is hosted on a public GitHub repository at https://github.com/laetichou/FeS_Em_Prediction.git. For any updates or issues, interested parties may contact the corresponding author at l.a.e.guerin@student.tudelft.nl.

Model	Scaling	Feature Selection	Hyperparameters
Linear Regression	Yes	Yes	None
Elastic Net	Yes	Yes	$\alpha \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$, $l1_ratio \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$
SVR	Yes	Yes	$C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, \text{auto}, 0.001, 0.01, 0.1, 1\}$, $\epsilon \in \{0.01, 0.1, 0.2\}$, kernel = rbf
GPR	Yes	Yes	kernel $\in \{\text{Constant} \times \text{RBF} + \text{White}, \text{Constant} \times \text{RBF}, \text{RBF} + \text{White}\}$, $\alpha \in \{10^{-10}, 10^{-8}, 10^{-6}, 10^{-4}\}$
KNN	Yes	Yes	$n_neighbors \in \{3, 5, 7, 9, 11, 15\}$, weights $\in \{\text{uniform}, \text{distance}\}$, metric $\in \{\text{euclidean}, \text{manhattan}, \text{minkowski}\}$
Random Forest	No	No	$n_estimators \in \{100, 200\}$, $max_depth \in \{5, 7, 10\}$, $min_samples_split \in \{2, 5, 10\}$, $min_samples_leaf \in \{1, 2, 4\}$, $max_features \in \{\text{sqrt}, \text{log2}\}$
XGB	No	No	$n_estimators \in \{100, 200\}$, $learning_rate \in \{0.05, 0.1\}$, $max_depth \in \{3, 5\}$, $min_samples_split \in \{2, 5\}$, $min_samples_leaf \in \{1, 2, 4\}$, $subsample \in \{0.8, 1.0\}$

Table E.3: **Machine learning models and their configurations used in the redox potential prediction pipeline.** α in Elastic Net and GPR controls regularization strength, penalizing large coefficients to prevent overfitting (higher values increase penalty). **kernel** in SVR and GPR defines the transformation (e.g., 'rbf' [radial basis function] for nonlinear patterns, 'Constant $\times$ RBF' for Gaussian processes). γ in SVR adjusts the influence radius of training points (smaller values broaden it). ϵ in SVR sets the margin of tolerance for errors. **n_neighbors** in KNN specifies the number of nearest points considered, with **weights** ('uniform' or 'distance') and **metric** ('euclidean', 'manhattan', 'minkowski') affecting distance weighting. **max_depth** and **min_samples_split** in Random Forest and XGB limit tree depth and node splits to control complexity, while **subsample** in XGB fractions the training data per tree.